## Projekt: Datenmodellierung

## Erarbeitungs-/Reflexionsphase PHASE_2

Für die Umsetzung wurde das relationale Datenbankmanagementsystem MySQL gewählt. Die Entscheidung basiert auf der hohen Performance bei Lesezugriffen, die insbesondere für E-Commerce-Systeme von Bedeutung ist, sowie auf der exzellenten Portabilität. Als Open-Source-Lösung ermöglicht MySQL zudem kosteneffiziente Skalierbarkeit ohne Vendor-Lock-in. In der Praxis überzeugt das System durch seine weite Verbreitung und eine ausgereifte Dokumentation.

Das Setup umfasst die MySQL-Version 9.6.0. Die technische Implementierung sowie die Verwaltung der Datenbank erfolgten vollständig über das MySQL-CLI, um eine präzise und fehlerfreie Ausführung der SQL-Statements sicherzustellen.

```
mysql> +---------+-----------------------------+
    -> | @@version | @@version_comment           |
    -> +---------+-----------------------------+
    -> | 9.6.0   | MySQL Community Server - GPL |
    -> +---------+-----------------------------+
```

**Abbildung 1** *MySQL-Version*

Für die visuelle Modellierung wurde dbdiagram.io eingesetzt, da dieses Tool eine präzise Definition MySQL-spezifischer Datentypen wie ENUM erlaubt. Aufgrund der Notationsvorgaben des Modellierungstools wird die Beziehung zwischen den Tabellen `orders` und `payment` grafisch als 1:n dargestellt. Um der konzeptionellen Anforderung einer strikten 1:1-Relation gerecht zu werden, wurde diese logische Integrität in der technischen Umsetzung durch gezielte Unique-Constraints sichergestellt.

Die 1:1-Beziehung zwischen Bestellung und Zahlung ergibt sich aus fachlicher Sicht daraus, dass jede Bestellung genau einmal bezahlt wird und weder Teilzahlungen noch Mehrfachzahlungen vorgesehen sind.
Die technische Umsetzung der Datenbank erfolgte ausschließlich über das CLI. Zunächst wurde das Schema angelegt und die Tabellenstruktur mittels `CREATE TABLE`-Statements definiert.

- **Fokus:** Präzises Setzen von PRIMARY- und FOREIGN KEY zur Sicherung der referentiellen Integrität.
- **Optimierung:** Konsequente Anwendung der Normalisierungsregeln zur Vermeidung von Redundanzen und zur Schaffung einer konsistenten Analysebasis.
- Die physische Umsetzung (siehe Abbildung 2) nutzt `NOT NULL` für Pflichtfelder wie E-Mails, um Datenlücken zu vermeiden. `UNIQUE`-Constraints sichern die Eindeutigkeit von Kundenaccounts. FOREIGN KEYS garantieren die referentielle Integrität, sodass beispielsweise keine Zahlungen ohne zugehörige Bestellung existieren können.

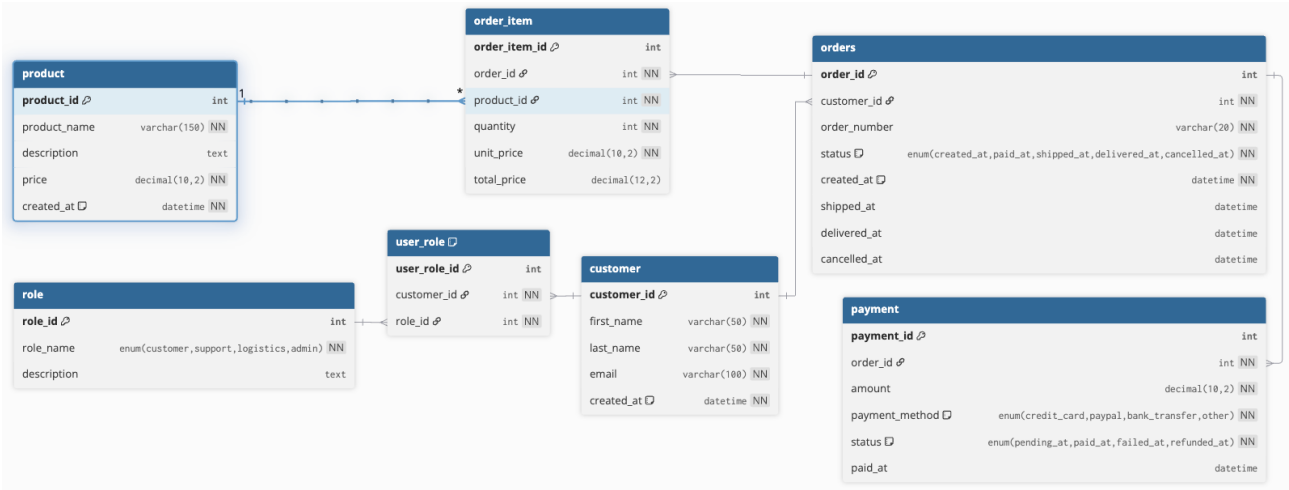


**Abbildung 2** *Physisches Entity-Relationship-Modell des E-Commerce-Systems*

### Hinweis zur Modellierung:

Die Relation zwischen den Tabellen `orders` und `payment` ist fachlogisch als 1:1-Beziehung definiert. Die toolbedingte Darstellung als 1:n wird in der technischen Implementierung korrigiert.

## Seed-/Testdaten & Abfragen

Nach der Schemadefinition wurden die Seed- und Testdaten manuell durch direkte Eingabe der SQL-Statements über das CLI eingebracht. Ziel war es, die unmittelbare Verarbeitung sowie die Integrität der Datensätze während des gesamten Prozesses zu validieren.

- **Vorgehen:** Einsatz gezielter Test-Statements, die absichtlich Integritätsverletzungen auslösen, um die korrekte Funktionsweise der definierten Constraints zu überprüfen.
- **Nachweis:** Die ausgegebenen Fehlermeldungen (z.B. `ERROR 1062: Duplicate entry`) belegen, dass die `Unique`-Constraints, etwa für E-Mail-Adressen oder Rollenzuweisungen, zuverlässig greifen.
- **Ergebnis:** Eine abschließende Statistik-Abfrage bestätigt die erfolgreiche Befüllung aller sieben Tabellen.

Der Ausschnitt aus dem Testprotokoll zeigt exemplarisch, wie das System eine doppelte E-Mail-Adresse korrekt abweist.

```
mysql> INSERT INTO customer (first_name,last_name,email) VALUES ('Max','Mustermann','max@example.com');
ERROR 1062 (23000): Duplicate entry 'max@example.com' for key 'customer.email'
```

**Abbildung 3** *Validierung der Datenintegrität durch Constraint-Prüfung*

Der Auszug aus der Status-Abfrage zeigt die erfolgreiche Verifizierung der Migration.

```
mysql> SELECT 'Kunden' AS Tabelle, COUNT(*) AS Anzahl FROM customer UNION
    -> SELECT 'Produkte', COUNT(*) FROM product UNION
    -> SELECT 'Bestellungen', COUNT(*) FROM orders UNION
    -> SELECT 'Bestellpositionen', COUNT(*) FROM order_item;
+-------------------+--------+
| Tabelle           | Anzahl |
+-------------------+--------+
| Kunden            |      7 |
| Produkte          |      6 |
| Bestellungen      |      6 |
| Bestellpositionen |      7 |
+-------------------+--------+
4 rows in set (0,001 sec)
```

**Abbildung 4** *Statusbericht der erfolgreichen Datenmigration*

## Reporting-Layer (2-Views)

Zur Analyse wurden Berichtsebenen mittels SQL-Views implementiert. Diese abstrahieren die Komplexität der zugrunde liegenden JOIN-Operationen und ermöglichen einen einfachen Zugriff für den Endanwender.

### Operatives Reporting (Monitoring des Auftragsflusses)

- **Funktion:** Die View `view_order_status_report` aggregiert Prozesszustände der Bestellungen in Echtzeit.
- **Technik:** Durch die Gruppierung nach Status sowie die Sortierung mittels `FIELD()`-Funktion wird der logische Workflow (`created_at → paid_at → shipped_at → delivered_at`) korrekt abgebildet.
- **Nutzen:** Die View ermöglicht eine unmittelbare Identifikation der unterschiedlichen Prozessphasen.
  - Prozess-Monitoring: Gibt es Verzögerungen bei den offenen Bestellungen?
  - Erfolgsquote: Wie viele Bestellungen wurden bereits erfolgreich abgeschlossen?
  - Problemidentifikation: Gibt es ungewöhnlich viele stornierte Aufträge?

### Definition: view_order_status_report

```
mysql> SHOW CREATE VIEW view_order_status_report;
+-------------------------+-----------------------------
```

### Auswertung: Anzahl Bestellung nach Bestellstatus

```
mysql> SELECT * FROM view_order_status_report;
+---------------+---------------------+
| Bestellstatus | Anzahl_Bestellungen |
+---------------+---------------------+
| paid_at       |                   3 |
| shipped_at    |                   1 |
| delivered_at  |                   1 |
| cancelled_at  |                   1 |
+---------------+---------------------+
4 rows in set (0,001 sec)
```

**Abbildung 6** *Output des operativen Bestellstatus-Berichts*

### Management Reporting – Umsatzanalyse pro Kunde

Als zweite Berichtskomponente wurde eine View zur Analyse der Kundenwertigkeit implementiert.

- **Funktion:** Die View `view_customer_revenue` verknüpft Kunden-, Bestell- und Zahlungsdaten zur Ermittlung des Gesamtumsatzes pro Kunde.
- **Technik:** Einsatz mehrerer `JOIN`-Operationen sowie der Aggregatfunktion `SUM()`, sortiert nach absteigendem Umsatz.
- **Nutzen:** Identifikation umsatzstarker Kunden zur Ableitung gezielter Marketingmaßnahmen und Treueprogramme.
- **Performance-Check:** Mithilfe einer `EXPLAIN`-Analyse wurde das Optimierungspotenzial der Reporting-Views überprüft. Ziel war die Sicherstellung einer effizienten Index-Nutzung sowie der Vermeidung von Full Table Scans.

### Definition: view_customer_revenue

```
mysql> SHOW CREATE VIEW view_customer_revenue;
+----------------------+-----------------------
```

## Auswertung: Kunde nach Gesamtumsatz

```
mysql> SELECT * FROM view_customer_revenue;
+-------------+----------------+--------------+
| customer_id | Kunde          | Gesamtumsatz |
+-------------+----------------+--------------+
|           1 | Max Mustermann |      1325.50 |
|           4 | Laura Meier    |       180.00 |
|           2 | Anna Müller    |       150.00 |
|           3 | Peter Schmidt  |       150.00 |
|           6 | Lisa Klein     |        51.00 |
|           5 | Tom Fischer    |        10.00 |
+-------------+----------------+--------------+
6 rows in set (0,000 sec)
```

**Abbildung 8** *Ergebnis der strategischen Top-Kunden-Auswertung*

## EXPLAIN-Analyse

```
mysql> EXPLAIN SELECT * FROM view_customer_revenue;
+-------------------
```

## Reflexion und Fazit

Die erfolgreiche Umsetzung dieses Datenbankprojekts verdeutlicht den vollständigen Prozess von der theoretischen Modellierung bis zur praktischen Anwendung in einem E-Commerce-Kontext. Durch die konsequente Einhaltung der dritten Normalform wurde eine stabile Grundlage für konsistente und redundanzfreie Daten geschaffen, die insbesondere für den Reporting-Layer von zentraler Bedeutung ist. Während der Befüllungsphase bestätigten gezielt herbeigeführte Integritätsverletzungen die Wirksamkeit der physischen Constraints und gewährleisten eine hohe Datenqualität.

Die finale Implementierung der zwei Reporting-Views unterstreicht den geschäftlichen Mehrwert der technischen Struktur:

- Die operative View (`view_order_status_report`) ermöglicht eine effiziente Überwachung des Bestellflusses sowie die frühzeitige Identifikation logistischen Engpässe.
- Die strategische View (`view_customer_revenue`) transformiert Rohdaten in aussagekräftige Kennzahlen zur Kundenbewertung und unterstützt fundierte Marketingentscheidungen.

Insgesamt zeigt das Projekt, wie durchdachte Datenstrukturen gezielt in geschäftsrelevante Informationen überführt werden können, um sowohl operative Exzellenz als auch strategisches Wachstum zu fördern.